PAL API Guide

This guide combines a first working model and practical modeling workflows with the detailed behavioral contracts of PAL's public API. Start with the worked sections, then use the API sections below to check semantics, constraints, and examples. For current public signatures and method descriptions, use the API Reference for signatures and method descriptions. For quick syntax, use the Cheatsheet. For implementation structure and design rationale, see PAL Architecture and Design.

Getting started and worked modeling patterns

The sections below walk from the model loop through representation choices, neighborhoods, agent updates, synchronous population/field updates, compiled code, visualization, and reproducibility. These examples make the API contracts concrete; the detailed API sections that follow are the reference for exact behavior.

1. The model loop

A PAL model commonly has three parts: **Setup** creates the initial state, **Step** updates it, and **Draw** renders it. PAL does not require a model class or impose a scheduler; you choose the update order and when to render.

```
import PythonAutomataLibrary as pal

@pal.njit(cache=True)
def Step(pop: pal.PopGrid):
    # Apply a synchronous increment to every site.
    for i in range(len(pop)):
        pop.Add(1, i)
    pop.Update()

def main():
    pop = pal.NewPopGrid((40, 40))
    for _ in range(10):
        Step(pop)
    print(pop.GetPop())

if __name__ == "__main__":
    main()
```

The grid contains 1,600 sites. Each site receives one increment per timestep for 10 timesteps, so every site ends at 10 and `GetPop()` returns `16000`. `Add` queues changes; `Update` commits them together. This separation prevents an update at one site from changing the state read at another site during the same timestep.

PAL uses **safe mode** by default. Develop and validate your model in this mode. If profiling shows that validation overhead matters, call `pal.FastMode()` before creating any PAL objects to remove selected performance-sensitive checks.

2. Choose the right representation

Choose the representation that matches the state your model needs to preserve. A `Grid` stores one typed value per site; an `AgentGrid` stores identifiable agents and can hold per-agent properties; a `PopGrid` stores integer counts without individual identities; and a `PDEgrid` stores a scalar field with transport operations. A model can combine them—for example, agents can consume a `PDEgrid` field while a separate `PopGrid` tracks counts.

Use a tuple of dimensions when constructing a spatial grid. Negative dimensions mean periodic wrapping on that axis, so `(-40, 40)` wraps x but not y. `AgentGrid` also supports an empty dimension tuple for a nonspatial population.

Choosing a representation by scale

Use individual agents when identity, history, or per-agent properties affect behavior. If individuals are interchangeable within each site, a `PopGrid` can replace many agent handles with one integer count per site, often reducing update cost. Its optional `capacity` limits each site, not the total population. Use a `Grid` for nonpopulation site attributes such as terrain or a fixed mask, and a `PDEgrid` for a scalar field that changes through diffusion or advection. A hybrid model can combine representations, but must define how they exchange state.

Choose a representation according to what the model must preserve, not just the number of sites. Replacing agents with counts discards individual identity and history; replacing counts with a continuous field can discard integer and stochastic behavior. These are modeling assumptions, not merely implementation details.

3. Coordinates and neighborhoods

A grid has integer lattice sites and may also store agents at continuous positions. A **site index** `i` is an integer identifying a lattice cell; lattice coordinates such as `(x, y)` identify that cell on each axis. An agent's continuous coordinates are different: lattice site `(x, y)` spans continuous positions `[x, x+1) × [y, y+1)`, and its center is `(x+0.5, y+0.5)`. Thus an agent created or moved with `NewAgentSQ(x, y)` or `MoveSQ(..., x, y)` has integer site coordinates `XSQ=x`, `YSQ=y`, but continuous coordinates `X=x+0.5`, `Y=y+0.5`. An agent placed with `NewAgent` or `Move` instead keeps the supplied continuous coordinates while its `XSQ` / `YSQ` values identify the containing lattice site. In 2D, `ToI(x, y)` converts lattice coordinates to a linear site index, and `ItoX(i)` and `ItoY(i)` convert that index back to lattice coordinates. Do not confuse the linear site index, lattice coordinates, and continuous agent coordinates. Grid geometry is exposed as attributes such as `xDim`, `yDim`, `nDims`, and `wrapX`, not methods.

PAL provides `MooreHood`, `VonNeumannHood`, and `CircleHood`. They return relative integer offsets. `grid.Hood(hood, x, y)` maps those offsets to **lattice coordinates** (a scalar x in 1D, coordinate tuples in 2D/3D), wrapping periodic axes and omitting sites outside nonperiodic axes. It preserves the order and duplicates of the supplied offsets. `grid.Box(...)` instead iterates coordinates in a half-open rectangular region; in 2D it also yields `(x, y)` tuples, not linear indices. A box wider than a wrapped axis can also visit a site more than once. This distinction matters when using the results for indexing versus coordinate-based operations.

4. Individual agents

An agent is an integer handle owned by an `AgentGrid`, distinct from both its lattice site and its continuous position. Create it with `NewAgentSQ` for a lattice site or `NewAgent` for a continuous position. `NewAgentSQ` and `MoveSQ` accept either a linear site index or lattice coordinates; `NewAgent` and `Move` use continuous coordinates. An agent can move within a site without changing its lattice site. Remove it with `Dispose`.

`agents.All()` returns a snapshot of living handles, so creating or disposing agents does not change which handles are in that snapshot. On a nonstackable grid, safe mode rejects attempts to place two agents at the same site. Use `isStackable=True` only when multiple occupancy is part of the model.

Iterating nearby agents

Use `AgentsAt(x, y)` to visit the agents occupying one lattice site. For a continuous search around a point, `AgentsInRadius(rad, x, y)` yields `(agent, dx, dy, distSq)` in 2D. The displacement components account for periodic wrapping and `distSq` is squared distance. In 1D the tuple is `(agent, dx)`; in 3D it is `(agent, dx, dy, dz, distSq)`. Pass `exclude=agent` to omit an agent from its own neighborhood search.

Both iterators work in Python and inside `@pal.njit` loops. Python iteration materializes matching handles first, but the grid must not be structurally modified during that iteration. Safe mode checks for structural changes; fast mode omits this check. If the loop creates or disposes agents, iterate over a snapshot from `agents.All()` instead.

For example, this Python-side search finds the agent across a periodic x boundary:

```
import PythonAutomataLibrary as pal

agents = pal.NewAgentGrid((-10, 10))
agent = agents.NewAgent(9.0, 2.0)
nearby = list(agents.AgentsInRadius(1.5, 0.0, 2.0))
assert len(nearby) == 1
found, dx, dy, distSq = nearby[0]
assert found == agent
assert dx == -1.0 and dy == 0.0 and distSq == 1.0
assert list(agents.AgentsInRadius(1.5, 0.0, 2.0, exclude=agent)) == []
```

Agent birth/death timestep

`Examples/Agents/SaveLoadModel.py` is a complete agent-based model using a nonstackable 2D `AgentGrid`. Its compiled `Step` iterates `grid.All(shuffle=True)`, removes an agent on a death draw, and otherwise attempts reproduction on a second draw. To reproduce, it scans the Moore neighborhood, stores empty site indices in a reusable `IList`, and creates an agent at a randomly selected empty site.

The distinction between **snapshot membership** and **current occupancy** is important. `All()` determines which agents get a turn at the start of the step, so newborns do not act until the next step. However, each parent's neighborhood query sees the current grid; births and deaths earlier in the step can therefore change later agents' options. Shuffling changes the update order and can change outcomes even from the same initial occupancy. This is a sequential stochastic update, unlike the queued, simultaneous `PopGrid.Add` / `Update` pattern described below.

The example's `Setup`, `Step`, and `Draw` functions are `@pal.njit(cache=True)` functions. The Python `main` loop owns the window, timestep, and checkpoints; compiled functions handle repeated model operations. Use the source example for the complete runnable program rather than duplicating its graphics and checkpoint lifecycle here.

5. Simultaneous population and field updates

`PopGrid` and `PDEgrid` distinguish current state from pending changes. Direct indexing changes the current state immediately; `Add` queues a delta, and `Update` applies pending changes. `Reset` clears both current state and pending changes.

A `PDEgrid` also provides diffusion and advection. Set the timestep and spatial spacings with `SetTimeSpaceStep(dt, dx, dy, dz)` before transport operations. Cartesian, interface-based, masked, ADI, and radial diffusion variants are available; choose the method that matches the model's geometry and numerical assumptions. Cartesian boundaries may be uniform or vary across a face; wrapped axes are periodic instead. Validate timestep stability and boundary behavior in safe mode before optimizing.

Advection versus diffusion example

`Examples/Diffusibles/ReactionDiffusion2D.py` compares two `PDEgrid` fields on the same 20 × 10 lattice. Each timestep sets a short vertical source at `x=10` in both fields. One field then calls `Advection(vx=0.01, vy=0.01, xMaxBC=0, yMaxBC=0)`; the other calls `DiffusionADI(0.01, xMaxBC=0, xMinBC=0)`. Each transport call is followed by its own `Update()`.

The example illustrates two different processes: advection transports the field with a velocity, whereas diffusion spreads it down concentration gradients. Boundary values also enter the two methods differently. Diffusion uses the boundary gradient; advection uses a boundary value only for inflow and uses the interior upwind value for outflow. The explicit boundary arguments are part of the model, not cosmetic rendering settings. `Draw` maps each field's current values through `ColorScale` into separate pixel windows. As in the agent example, `Step` and `Draw` are compiled, while `main` manages windows and iteration.

This example is qualitative, **not** a convergence or mass-conservation test. It continually resets source cells and uses different boundary conditions for the two fields. For quantitative validation, remove the source, use equivalent boundary conditions, and check mass balance and convergence across grid spacings and timesteps.

A hand-checkable field gradient

`Examples/Diffusibles/Gradient2D.py` provides a simpler numerical check than a transport simulation. It assigns `grid[x, y] = y / grid.yDim` on a 10 × 10 `PDEgrid`. The centered finite differences at `(5, 5)` are therefore

```
import PythonAutomataLibrary as pal

field = pal.NewPDEgrid((10, 10))
for x in range(10):
    for y in range(10):
        field[x, y] = y / 10

dx = (field[6, 5] - field[4, 5]) / 2
dy = (field[5, 6] - field[5, 4]) / 2
assert abs(dx) < 1e-6
assert abs(dy - 0.1) < 1e-6
```

The field is constant in x and linear in y, so its expected discrete gradients are 0 and 0.1. The assertions allow floating-point storage and arithmetic error. This verifies indexing and a finite-difference calculation without introducing boundary conditions or timestep integration. `Examples/Diffusibles/Gradient2D.py` performs the same calculation inside `@pal.njit`.

Numerical update order

Update order is part of the model. If births and deaths in a `PopGrid` must be simultaneous, queue all changes before calling `Update()`. Reads during this phase see the same starting population. Direct assignment to `pop[x, y]`, by contrast, changes what later calculations read. Mixing immediate assignments and queued changes is valid, but the order then affects the model.

For a `PDEgrid`, transport routines accumulate changes that become current at `Update()`. Check timestep stability and convergence by varying the timestep and grid spacing. Safe-mode checks do not replace numerical validation.

Consider two sites with populations 10 and 0. If a transfer moves 5 from site 0 to site 1, queue -5 and +5 before a single `Update()`; the result is (5, 5) and total population remains 10. Applying one change immediately would alter what later calculations read and can make an intended simultaneous update order-dependent.

For a hand-calculable timestep, check the required invariants: counts remain nonnegative; internal transfers conserve total population; totals change only through explicit births, deaths, or boundary flux; and a supposedly simultaneous update is independent of site visitation order.

6. Compiled model code

Decorate substantive computational functions with `@pal.njit(cache=True)` so compiled code can be reused across runs. PAL expands some model constructs before passing the function to Numba, including neighborhood iteration and source-aware safe-

mode diagnostics. Annotate PAL arguments with their public types, such as `pal.AgentGrid` or `pal.PDEgrid`, for clarity and compiler support.

Keep model state in PAL objects and use `pal.Seed`, `pal.Random`, `pal.RandInt`, and `NewMultinomial` for reproducible PAL random sampling. Run a model in safe mode first. Fast mode removes selected checks; it does not make invalid indices, dead handles, or unstable numerical steps valid.

Testing a timestep

Test a model's update function on a grid small enough to inspect by hand, including an edge or wrapped boundary rather than only interior sites. Use invariants appropriate to the object: closed diffusion conserves mass within floating-point tolerance; agent moves preserve valid live handles and occupancy rules; population updates satisfy the conservation and nonnegativity checks from §5.

When a model uses both Python orchestration and compiled `@pal.njit` updates, test both paths; passing in Python does not establish compiled behavior. Include safe and fast runs in regression testing, but use safe-mode exceptions to diagnose invalid operations rather than expecting fast mode to detect them.

Random streams and repeatability

`pal.Seed(seed)` initializes PAL's random stream. `pal.Random()` draws a uniform value and `pal.RandInt(max)` draws an integer from `[0, max)`. PAL random draws from Python and compiled PAL code consume the same stream, so **the order of calls matters**. Repeating a run requires the same seed, initial state, model parameters, and random-call order; merely reseeding halfway through a simulation does not reconstruct the prior model state.

For comparisons between two algorithms, seed each run separately and record the seed. If one implementation makes additional random draws, subsequent results can diverge even when both start from the same seed. External random libraries (such as NumPy's generators) have independent state and must be seeded or checkpointed separately.

7. Drawing and output

`StartPixWindow(xDim, yDim, scale=1, title='PAL', headless=False)` returns `(pix, window)`. Write packed RGB colors such as `0xFF0000` to `pix[x, y]`, then call `window.Update()` to publish the frame. `window.Save(path, block=True)` waits for the image to be written; use `window.Close()` to release resources. The pixel buffer also supports linear indices, slices, and writes from compiled code. For off-screen images, use `headless=True` (requires the relevant output dependencies). GIF recording uses `window.StartGif(path, delay=100)`, `window.AddGifFrame(block=False)`, and `window.StopGif()`; update the window before capturing each frame.

`StartOpenGLWindow` provides geometric 2D/3D drawing rather than a writable pixel lattice. Use its drawing object for shapes and scene controls; its window uses the same update, image/GIF output, and close lifecycle described above. `headless=True` is available for off-screen rendering but still requires a working OpenGL backend. See the API Reference for the drawing-method inventory.

Rendering and simulation are separate. A model can run without a window, draw every timestep, or draw only selected checkpoints.

8. Checkpointing and reproducibility

PAL's native-backed model objects support Python pickling, so a long-running model can save its state and resume without retaining native pointers from the original process. A checkpoint must include **all** state needed for continuation: every grid, relevant parameters, the current timestep, and any additional state managed by the model. Saving only one grid does not preserve a

coupled simulation. Keep the code version and parameter configuration alongside the checkpoint so that results can be interpreted later.

A PAL object checkpoint does **not** serialize PAL's random stream. If exact stochastic continuation matters, the model must reconstruct the intended random state separately; external RNGs such as NumPy's are independent as well. Verify checkpoint behavior with a short uninterrupted run and a checkpoint-and-resume run rather than assuming object state alone reproduces the same future random draws.

A minimal checkpoint can be saved and resumed without opening a window:

```
import pickle
import PythonAutomataLibrary as pal

pop = pal.NewPopGrid((5,))
pop[1] = 10
pop.Add(7, 1)  # Pending change is part of the checkpoint.
state = {"pop": pop, "step": 3}
checkpoint = pickle.dumps(state)

restored = pickle.loads(checkpoint)
restored["pop"].Update()
assert restored["step"] == 3
assert restored["pop"][1] == 17
```

`Examples/Agents/SaveLoadModel.py` serializes a dictionary containing an `AgentGrid`, an `IList`, and the timestep, then restores all three before continuing. PAL object checkpoints do not include unrelated Python state or external RNG streams.

Core use

```
import PythonAutomataLibrary as pal
```

PAL starts in **safe mode**. Safe mode checks public operations and provides useful errors. To select fast mode:

```
pal.FastMode()
```

Call `FastMode()` before constructing any PAL model or window object. The first such construction locks the process-wide mode. Fast mode removes safety checks from performance-sensitive paths; it is intended for models already validated in safe mode.

Use `@pal.njit` for model computation:

```
@pal.njit
def Step(grid: pal.AgentGrid):
    ...
```

`pal.njit` wraps Numba `njit` and additionally transforms PAL model code. Among other PAL-specific compilation behavior, annotated PAL arguments allow safe-mode errors in compiled model code to report the model source line.

Construction

Function	Purpose
`NewGrid(dimensions, dtype)`	Typed lattice values.
`NewAgentGrid(dimensions, numAgentProps=0, isStackable=False)`	Discrete or continuous agents with optional per-agent properties.
`NewPopGrid(dimensions, capacity=None)`	Integer population counts on a lattice.
`NewPDEgrid(dimensions)`	Continuous scalar field with diffusion/advection operations.
`NewIList()`	Reusable integer query/list container.
`NewMultinomial(other=None)`	Binomial/multinomial sampler.

Spatial grids use one to three nonzero integer dimensions. Each signed dimension must fit the int32 range (excluding its minimum value), and the product of absolute axis lengths must not exceed the int32 maximum. Fractional, nonfinite, zero, oversized, or malformed dimension inputs raise `ValueError` during construction. A **negative dimension enables wrapping on that axis** while its absolute value gives the size. `AgentGrid` alone also accepts `()` for a zero-dimensional nonspatial population:

```
grid = pal.NewAgentGrid((100, 100))    # 100 x 100, no wrapping
grid = pal.NewAgentGrid((-100, 100))   # x wraps, y does not
grid = pal.NewPDEgrid((-100, -100))    # x and y wrap
```

`numAgentProps` is the number of per-agent floating-point properties. It must be a nonnegative integer fitting AgentGrid's int32 metadata budget (which also reserves entries for spatial dimensions); fractional, nonfinite, negative, or oversized values raise `ValueError`. `isStackable` accepts booleans or `0` / `1`; other values raise `ValueError`. When false, at most one agent occupies each lattice site. `PopGrid.capacity` is an optional nonnegative **per-site** population limit, not a limit on the total across the grid.

`NewMultinomial(other)` accepts another PAL Multinomial in the **same safety mode** and copies its solver configuration, not its current sampling state. An unrelated object raises `TypeError`; mixing safe and fast samplers raises `ValueError`. Create a new sampler without `other` to use the default solver.

Shared spatial API

`Grid`, `AgentGrid`, `PopGrid`, and `PDEgrid` deliberately share spatial conventions. Geometry metadata (`xDim`, `yDim`, `zDim`, `nDims`, `wrapX`, `wrapY`, `wrapZ`) is read as attributes; geometry operations such as `ToI(...)` and `Box(...)` are methods.

Geometry

Member	Meaning
`len(grid)`	Number of lattice sites.
`xDim`, `yDim`, `zDim`	Axis sizes. Missing axes are invalid in safe mode.
`nDims`	Number of spatial dimensions.

Member	Meaning
`wrapX`, `wrapY`, `wrapZ`	Whether each axis wraps.
`ToI(x, y=-1, z=-1)`	Convert coordinates to a linear lattice index.
`ItoX(i)`, `ItoY(i)`, `ItoZ(i)`	Recover coordinates from a linear index.
`InWrapX/Y/Z(value)`	Wrap an integer lattice coordinate on that axis, or return `-1` when outside a nonwrapped axis.

The same geometry names are used in 1D, 2D, and 3D. Supply only coordinates/axes that exist for the grid. `Grid`, `PopGrid`, and `PDEgrid` use `InWrapX/Y/Z` for lattice coordinates; `AgentGrid` distinguishes lattice `InWrapSQX/Y/Z` from continuous `InWrapX/Y/Z`.

Regions and neighborhoods

```
grid.Box(x1, x2)
grid.Box(x1, x2, y1, y2)
grid.Box(x1, x2, y1, y2, z1, z2)

grid.Hood(hood, x)
grid.Hood(hood, x, y)
grid.Hood(hood, x, y, z)
```

`Box` iterates coordinates in the half-open rectangular region `[x1, x2)`, `[y1, y2)`, `[z1, z2)` for the dimensions supplied. It yields an integer in 1D, `(x, y)` in 2D, and `(x, y, z)` in 3D. Bounds must be int32 integers, and the number of bound pairs must match grid dimensionality. On wrapped axes, out-of-range coordinates are wrapped; on nonwrapped axes, they are skipped. A box wider than a wrapped axis can therefore yield repeated sites. `Hood` maps relative offsets around a lattice position to the same coordinate forms, preserving offset order and duplicates; wrapped coordinates are mapped into the grid and nonwrapped out-of-range coordinates are omitted. Offset tuple lengths must match grid dimensionality. Inside `@pal.njit`, `unroll=True` requests compile-time neighborhood unrolling without changing the returned coordinates. `Box` and `Hood` are direct-iteration constructs in compiled code: top-level `break` or `continue` from their loop body is unsupported; use explicit `range` loops when that control flow is needed.

Built-in neighborhoods:

```
pal.MooreHood(dim, excludeCenter=False)
pal.VonNeumannHood(dim, excludeCenter=False)
pal.CircleHood(dim, rad, excludeCenter=False)
```

`MooreHood` contains offsets with each coordinate in `[-1, 1]`. `VonNeumannHood` contains the center and axis-adjacent offsets. `CircleHood` contains integer offsets whose Euclidean distance from the center is at most `rad`; `rad` may be noninteger, but must be finite and nonnegative. Set `excludeCenter=True` to omit the zero offset. All three constructors support dimensions 1–3 and reject other dimension counts with `ValueError`. Each returns a tuple of integer offset tuples; these describe relative positions, not site indices, until mapped through `grid.Hood(...)`.

Grid

```
grid = pal.NewGrid((xDim, yDim), dtype)
```

Supported dtypes are `bool`, signed/unsigned 8/16/32/64-bit integers, and 32/64-bit floats.

`Grid` supports linear indexing, coordinate indexing, and NumPy-like slices:

```
grid[i]
grid[x, y]
grid[x, y, z]
grid[:] = 0
region = grid[2:5, 3:8]
```

Slice reads return detached NumPy copies. Scalar and slice reads/writes work in Python and compiled PAL model code. In safe mode, invalid scalar indices and coordinates are rejected; fast mode assumes valid access. Safe mode validates bounds, dimensionality, and whether assigned values are representable by the Grid dtype: boolean grids accept only `0` / `1`; integer grids reject fractional, nonfinite, and out-of-range assignments; floating grids reject nonfinite or out-of-range assignments. Use a wider dtype when a calculation may exceed its current range. A detached slice copy is not a live view of the Grid.

AgentGrid

```
agents = pal.NewAgentGrid((xDim, yDim), numAgentProps=2)
```

PAL represents an agent by an integer handle. Agent properties are stored on the grid and accessed with `agents[agent, property]`.

Population and lifecycle

Operation	Meaning
`GetPop()`	Number of living agents.
`Alive(agent)`	Whether an agent handle is alive.
`NewAgentSQ(...)`	Create an agent at a lattice site.
`NewAgent(...)`	Create an agent at a continuous position.
`Dispose(agent)`	Remove an agent.
`All(shuffle=False)`	Snapshot of all living agents; optionally shuffled.

`All()` has **snapshot semantics**. The returned collection can be iterated while agents are created or disposed without changing the current iteration. `All(shuffle=True)` returns the same population in randomized order.

```
for agent in agents.All():
    if ShouldDie(agent):
        agents.Dispose(agent)
```

Position

Discrete/lattice position:

```
agents.I(agent)
agents.XSQ(agent)
agents.YSQ(agent)
agents.ZSQ(agent)
agents.MoveSQ(agent, x, y, z)
```

Continuous position:

```
agents.X(agent)
agents.Y(agent)
agents.Z(agent)
agents.Move(agent, x, y, z)
```

`NewAgentSQ` and `MoveSQ` accept either a linear lattice index or lattice coordinates in every spatial dimension. SQ placement uses the center of the selected cell: after `NewAgentSQ(x, y)`, for example, `XSQ(agent)==x` and `YSQ(agent)==y`, while `X(agent)==x+0.5` and `Y(agent)==y+0.5`. Continuous `NewAgent` and `Move` instead keep the supplied spatial coordinates, with `XSQ` / `YSQ` / `ZSQ` identifying the containing cell.

Occupancy and queries

```
agents.LastAgent(x, y, z)
agents.AgentsAt(x, y, z)
agents.counts[x, y, z]
```

`LastAgent` returns the most recently stacked agent at a site, or `-1` if the site is empty. Iterate with `for agent in agents.AgentsAt(x, y):` in Python or inside `@pal.njit`. Do not create, dispose, or move agents during direct iteration, including moves within the same lattice site. Safe mode detects these changes and raises `RuntimeError`; fast mode omits that check. In Python, `AgentsAt(i)` also accepts a linear site index, including for multidimensional grids; `AgentsAt(x, y)` and `AgentsAt(x, y, z)` use coordinates. This overload is distinct from the continuous-position `AgentsInRadius` query. `counts` exposes lattice occupancy counts and supports linear-site, coordinate, and slice indexing (`agents.counts[i]`, `agents.counts[x, y]`, or `agents.counts[:, :]`). It is a **read-only query view**, not a second mutable population grid. Slice queries return detached arrays of occupancy counts. A nonspatial (`dimensions=()`) AgentGrid has no lattice counts; querying `len(agents.counts)` raises `ValueError`.

By default an `AgentGrid` is not stackable: at most one agent may occupy a lattice site. Set `isStackable=True` when multiple agents per site are required. Safe mode rejects creation or movement into an occupied site on a nonstackable grid without changing model state.

Radius iteration

`AgentsInRadius(rad, x[, y[, z]], exclude=None)` iterates agents near a continuous point in Python or inside `@pal.njit`. The yielded tuple depends on dimensionality:

```
for agent, dx in agents1.AgentsInRadius(rad, x):
    ...
for agent, dx, dy, distSq in agents2.AgentsInRadius(rad, x, y):
    ...
for agent, dx, dy, dz, distSq in agents3.AgentsInRadius(rad, x, y, z):
    ...
```

The radius is inclusive: agents with distance exactly `rad` are included. Returned displacements point from the query center to the agent and use the shortest periodic displacement on wrapped axes; `distSq` is their squared Euclidean distance in 2D/3D. `exclude` omits a specified agent handle. As with Python-side `AgentsAt`, structural modification during iteration is unsupported, and the generation check is enabled only in safe mode.

Wrapping

Discrete coordinates:

```
InWrapSQX(value)
InWrapSQY(value)
InWrapSQZ(value)
```

Continuous coordinates:

```
InWrapX(value)
InWrapY(value)
InWrapZ(value)
```

Wrapped displacement:

```
DispWrapX(x1, x2)
DispWrapY(y1, y2)
DispWrapZ(z1, z2)
```

`InWrapSQ*` maps integer lattice coordinates into the corresponding axis; `InWrap*` does the same for continuous coordinates. On a nonwrapped axis, an out-of-range coordinate returns `-1`. `DispWrap*(x1, x2)` returns the shortest signed displacement from `x1` to `x2`, accounting for periodic boundaries.

PopGrid

```
pop = pal.NewPopGrid((xDim, yDim), capacity=None)
```

`PopGrid` stores integer population counts. Indexing and direct assignment modify the current population immediately.

```
pop[x, y]
pop[x, y] = 10
pop[:] = 0
```

Linear (`pop[:]`) and spatial (`pop[:, :]`) slice reads return detached `int64` NumPy arrays; slice assignment writes to the population. Population changes can instead be accumulated and applied together:

```
pop.Add(delta, x, y)
pop.Update()
```

`Add` changes only the pending delta. Until `Update()` , `GetPop()` , indexing, and `All()` still reflect the current population. A single `Update()` commits the queued changes together, avoiding order-dependent reads during a synchronous timestep. Direct assignments during that timestep still change current values immediately.

Operation	Meaning
`GetPop()`	Total population over all sites.
`All()`	Copy of the linear indices of currently nonzero sites.
`Reset()`	Clear current populations and pending changes.

`capacity=None` uses the int64 maximum as the per-site population cap. An explicit `capacity` must be a nonnegative int64 integer; invalid, fractional, nonfinite, or out-of-range values raise `ValueError` during construction. Capacity constrains each site independently; total population is tracked separately and must remain representable as a nonnegative int64. In safe mode, direct writes enforce these bounds immediately. `Add()` queues signed changes without applying them to the current counts; a queued change may exceed a site's remaining capacity until it is checked at `Update()` . `Update()` validates every resulting site count and the total population before applying any pending delta; if validation fails, current populations and pending deltas are unchanged. Fast mode assumes these constraints are satisfied.

PDEgrid

```
field = pal.NewPDEgrid((xDim, yDim))
```

`PDEgrid` stores a continuous `float32` scalar field. Direct indexing/assignment changes the current field immediately. Slice reads return detached `float32` NumPy arrays; array slice assignment writes to the field, but editing a previously read slice does not. Linear (`field[:]`) and spatial (`field[:, :]`) slices are supported. `Add(value, ...)` accumulates a pending delta; `Update()` applies pending changes simultaneously. `Reset()` clears both the field and pending changes.

Space and time

```
field.SetTimeSpaceStep(dt, dx, dy=1.0, dz=1.0)
field.Dt()
field.Dx()
```

```
field.Dy()
field.Dz()
```

Transport methods use these spacings. `Dy()` and `Dz()` apply only when those dimensions exist; safe mode rejects missing-dimension access. `dt`, `dx`, `dy`, and `dz` must be finite and positive in safe mode.

Diffusion

```
field.Diffusion(rateConstant, ...)
field.DiffusionMask(rateConstant, mask)
field.DiffusionField(rateConstants, ...)
field.DiffusionInterfaces(rateConstantsX, rateConstantsY=None, rateConstantsZ=None, ...)
field.DiffusionADI(rateConstant, ...)
field.DiffusionRadialCircle(rateConstant, outerBC=None)
field.DiffusionRadialSphere(rateConstant, outerBC=None)
```

`Diffusion` uses one constant diffusion rate. `DiffusionField` uses spatially varying cell-centered rates; each interior face uses the harmonic mean of its two adjacent cell rates, so a zero rate on either side blocks diffusion across that face. `DiffusionInterfaces` supplies the positive-face interface rate for each cell and axis directly; the negative face uses the neighboring cell's stored positive-face rate. `DiffusionMask` treats mask values `>= 0` as active and values `< 0` as blocked; diffusion occurs only between adjacent active cells, so the magnitude of a nonnegative mask value is irrelevant. `DiffusionADI` uses Crank-Nicolson in 1D, Peaceman-Rachford ADI in 2D, and Douglas-Gunn directional solves in 3D. `DiffusionRadialCircle` and `DiffusionRadialSphere` use radial shell finite-volume discretizations on a nonwrapped 1D grid with at least two cells. Cell `i` spans radii `i*dx` to `(i+1)*dx`, so cell 0 is the central disk/sphere and symmetry gives zero flux through its inner face. `outerBC` optionally sets the exterior concentration at the outer face.

Cartesian diffusion methods accept optional `xMinBC`, `xMaxBC`, `yMinBC`, `yMaxBC`, `zMinBC`, and `zMaxBC` fixed boundary values. A boundary may be a scalar applied uniformly across the face or a C-contiguous `float32` array with one value per site on that face; for example, an x-face array has `yDim*zDim` values in 3D. On a nonwrapped face, `None` means no flux through that face; wrapped axes are periodic and do not accept explicit boundary conditions. Field/interface arrays must match the grid geometry and dimensionality. Safe mode validates boundary size/type/finite values, explicit-scheme stability, and other inputs before changing pending state.

Advection

```
field.Advection(vx, vy=0.0, vz=0.0, ...)
field.AdvectionField(xVels, yVels=None, zVels=None, ...)
field.AdvectionInterfaces(xVels, yVels=None, zVels=None, ...)
```

Advection uses a conservative first-order upwind discretization. `Advection` uses constant velocity components. `AdvectionField` uses cell-centered velocity fields and averages adjacent values at interior faces. `AdvectionInterfaces` supplies positive-face velocities directly; the negative face uses the neighboring cell's stored positive-face velocity. Supply velocity components only for dimensions that exist. The same scalar-or-face-array Cartesian boundary arguments used by diffusion are available: `None` closes a nonwrapped face to advective flux, while supplied values provide the exterior concentration used for inflow. Safe mode checks the CFL condition before accumulating changes.

IList

```
items = pal.NewIList()
```

`IList` is PAL's reusable integer query/list container. Stored values are nonnegative int32 integers in the supported contract.

Operation	Meaning
`Append(i)`	Append a nonnegative int32 value; returns the IList for chaining.
`Clear()`	Remove all entries; returns the IList for chaining.
`Random()`	Return a random entry.
`Shuffle()`	Shuffle entries in place; returns the IList for chaining.
`All()`	Return a detached copy.
`Iter()`	Return a no-copy/live iterable view.
`len(items)`, `items[i]`	Length/index access.

Use `Iter()` for one-pass iteration when a copy is unnecessary. Use `All()` when the returned values must remain independent of subsequent IList changes.

Random numbers and Multinomial

```
pal.Seed(seed)
pal.Random()       # uniform float
pal.RandInt(max)  # integer in [0, max)
```

`Seed` accepts integral seeds in the uint64 domain (`0` through `2**64 - 1`); invalid or nonfinite inputs raise `ValueError`. `RandInt(max)` requires a positive integer in the int64 domain and returns an integer in `[0, max)`; zero, negative, fractional, nonfinite, or out-of-range bounds raise `ValueError`. `Random()` returns a uniform floating-point draw. PAL random functions share one seeded stream across Python and compiled PAL calls, so reseeding reproduces the same call sequence across that boundary. The stream is call-order dependent: adding or removing a random draw changes later results. This does not seed NumPy's separate RNG.

For binomial/multinomial sampling:

```
multi = pal.NewMultinomial()
x = multi.Binomial(n, p)

multi.Setup(n)
a = multi.Sample(pA)
b = multi.Sample(pB)
# remaining count/probability mass stays in the sampler
```

`Setup(n)` initializes a multinomial sequence with `n` trials and probability mass 1 and is chainable. Each `Sample(p)` takes that category's probability in the original multinomial distribution, draws from the remaining trials using the corresponding conditional probability, then removes the sampled count and `p` from the remaining state. Thus successive calls such as `Sample(0.2)` and `Sample(0.3)` represent categories with probabilities 0.2 and 0.3, not 0.2 followed by 0.3 of the remainder; any unrequested probability mass remains unsampled. In safe mode cumulative requested probability cannot exceed 1. `Binomial(n, p)` performs an independent binomial draw and does not consume the current multinomial sequence. `NewMultinomial(other)` copies solver configuration but starts with fresh sampling state.

Visualization

PAL provides a pixel renderer for lattice models and an OpenGL renderer for 2D/3D geometry. Rendering runs separately from model computation.

PixWindow

```
pix, window = pal.StartPixWindow(xDim, yDim, scale=1, title="PAL", headless=False)
```

`pix` is a write-oriented pixel buffer, not a general-purpose numeric `Grid`: it supports assignment but does not expose a public pixel-read API. Assign integer RGB colors by coordinate or linear index:

```
pix[x, y] = 0xFF0000
pix[i] = 0x00FF00
```

It also provides `xDim`, `yDim`, `ToI(x, y)`, `ItoX(i)`, `ItoY(i)`, and `len(pix)`.

The window lifecycle is:

```
window.Update()
window.Save(path, block=False)
window.StartGif(path, delay=100)
window.AddGifFrame(block=False)
window.StopGif()
window.IsOpen()
window.Close()
```

`Update()` publishes the current pixel buffer to the renderer. `StartGif()` begins a GIF recording, each `AddGifFrame()` appends the currently rendered frame, and `StopGif()` finalizes the file. `block=True` on asynchronous output operations waits for that requested output to complete before returning; call `StopGif()` before relying on the GIF file as complete.

OpenGLWindow

```
draw, window = pal.StartOpenGLWindow(xDim, yDim, zDim=None,
                                     width=800, height=800,
                                     title="PAL", headless=False)
```

Draw geometry with:

```
draw.Circle(rad, color, x, y, z=0.0)
draw.Box(xLen, color, x, y, z=0.0, yLen=None, zLen=None)
draw.BoxSQ(color, x, y, z=0.0)
draw.Line(width, color, x1, y1, x2, y2, z1=0.0, z2=0.0)
```

Window controls include `Borders(width, color)`, `Camera(x, y, z, yaw=None, pitch=None)`, `Background(color)`, `Clear()`, `Update()`, `IsOpen()`, `Save(path, block=False)`, `StartGif(path, delay=100)`, `AddGifFrame(block=False, timeout=30)`, `StopGif(timeout=30)`, and `Close()`. The GIF lifecycle is the same as for `PixWindow`: start recording, append rendered frames, then stop to finalize the file.

Colors are packed integer RGB values such as `0xFF0000`. `pal.ColorScale(colors, value)` interpolates RGB channels across a sequence of colors for normalized `value`; values outside `[0, 1]` clamp to the endpoint colors.

Set `headless=True` for off-screen image/GIF generation. OpenGL headless rendering requires a supported standalone backend such as EGL.

Use `pal.AwaitWindows()` when model execution should wait for PAL windows to finish their lifecycle.

Safe and fast contracts

Safe and fast objects expose the same modeling API. PAL starts in safe mode. To opt into fast mode, call `pal.FastMode()` **before constructing any PAL model or window object**. The first such construction locks the process-wide mode selection; a later `pal.FastMode()` raises `RuntimeError`. Run safe and fast comparisons in separate processes rather than trying to switch modes mid-run. Safe mode validates public operations and is the development/default mode; fast mode assumes the model obeys those contracts and removes performance-sensitive checks. Examples of safe-mode checks include lattice bounds and dimensionality, dead-agent and occupancy errors, PopGrid overflow, and PDE field/interface array shape or dimensionality. Invalid operations in fast mode are outside the supported contract rather than an alternative error-handling API.

Use safe mode until the model is correct, then benchmark fast mode. Do not rely on safe-mode exceptions as model control flow.

Saving PAL objects

PAL native-backed model objects support standard Python pickling. Checkpoints preserve the state needed to resume the object rather than process-local pointers: AgentGrid geometry, occupancy, agents and properties; PopGrid geometry, capacity, current counts and pending deltas; PDEgrid geometry, time/space steps, current field and pending deltas; IList contents; and a Multinomial's current `Setup` / `Sample` remainder. Safe/fast identity is also restored. This means a PopGrid or PDEgrid may be pickled before `Update()` without losing its queued changes.

Choosing the right structure

Need	PAL structure
One typed value per lattice site	`Grid`
Individual agents with identity/properties	`AgentGrid`
Integer counts without individual identity	`PopGrid`

Need	PAL structure
Continuous diffusing/advecting field	`PDEgrid`
Reusable integer query result	`IList`
Binomial/multinomial draws	`Multinomial`

These structures can be composed in one `@pal.njit` model step. For example, agents can query a field, queue population changes, and then update the population and field at explicitly chosen points in the timestep. The order of those commits determines which state subsequent operations read.